

Datové struktury a algoritmy

Přednáška 3, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Administrativa a prezentace

- Administrativa: pozor na zbytečné penalizace u prezentací!
 - Pozdní rezervace (4) nebo odevzdání (1)
 - Od 16.10. nebudu rezervovat žádné téma bez relevantního zdůvodnění!
- Prezentace
 - Pole jako základní datový typ jazyka C++ (2)
 - Základní přehled a rozdělení ADT (1)
 - Jednosměrný spojový seznam (2)
 - Obousměrný spojový seznam (2)
 - Časová složitost algoritmů, přehled (1)
 - Rekurze (2)

Dnešní témata

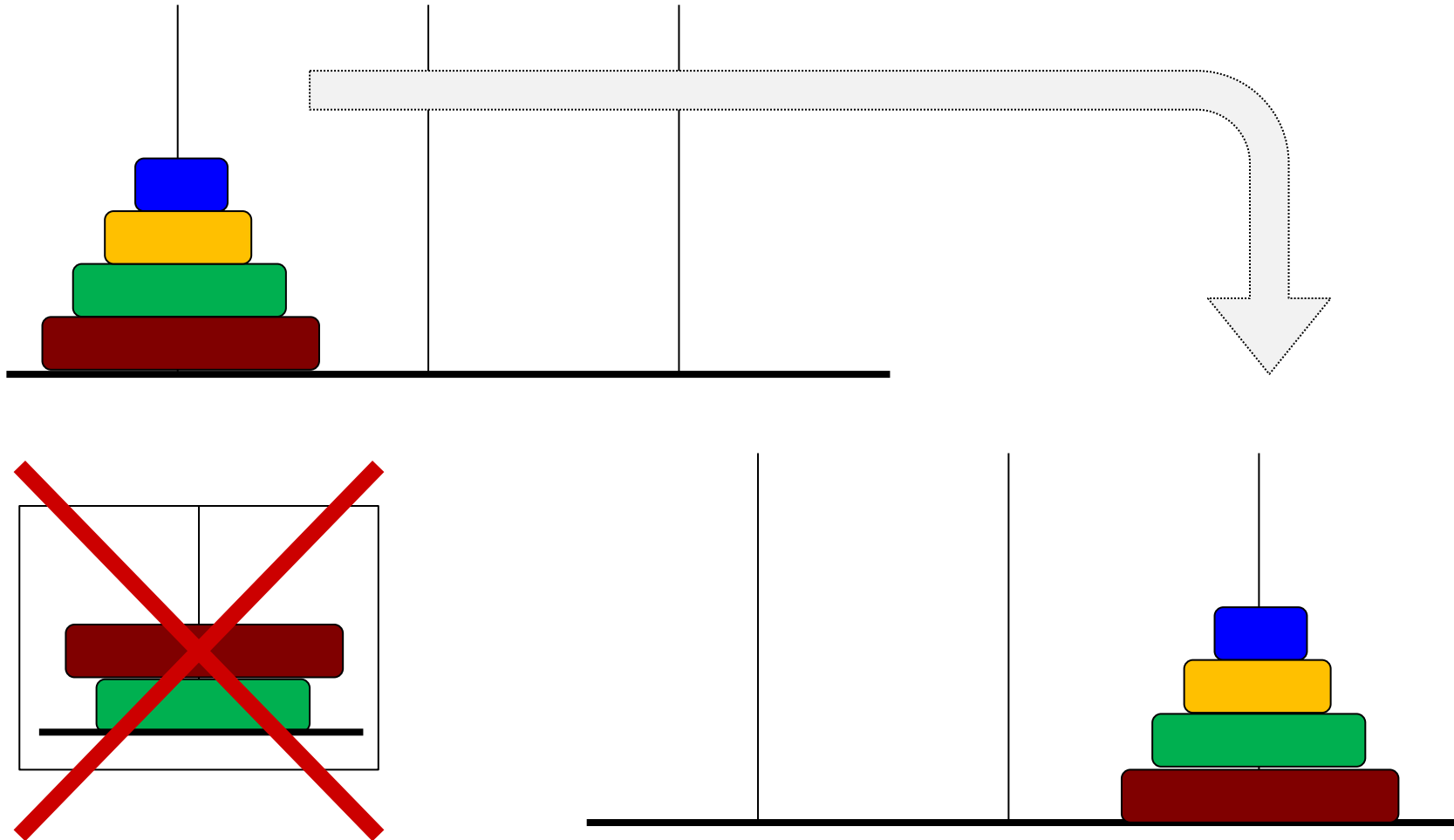
- Spojový seznam (dokončení z PP2)
 - Doplnující poznámky k PP1
 - Implementace ADT *List*
 - Odkazy na vizualizační aplikace
 - Demonstrace programového řešení jedno- a obousměrného spojového seznamu
- Výpočetní složitost, úvodní úvahy (v Moodle **téma č. 2**)
- Bonifikovaný kvíz (dojde-li na něj): Výpočetní složitost složených funkcí

Složitost, motivační příklad 1: List papíru

Představte si, že máte běžný kancelářský papír formátu A3 o síle 0,1 mm, který překládáte vždy na polovinu. Jaká bude celková tloušťka výsledného formátu, pokud by se vám ho podařilo takto přeložit 42x?

- 4,2 mm = $0,1 \times 42$
- 164 cm = výška člověka
- 105 m = délka fotbalového hřiště
- 215 km = vzdálenost Praha - Brno
- 439 805 km = vzdálenost větší než Země – Měsíc

Složitost, motivační příklad 2: Hanojské věže



Hanojské věže, algoritmizace

- Rekurzivní řešení:

(Zdroj: <https://www.cs.cmu.edu/~cburch/survey/recurse/hanoiimpl.html>)

```
FUNCTION MoveTower(disk, source, dest, spare):  
  IF disk == 0, THEN:  
    move disk from source to dest  
  ELSE:  
    MoveTower(disk - 1, source, spare, dest)  
    move disk from source to dest  
    MoveTower(disk - 1, spare, dest, source)  
  END IF
```

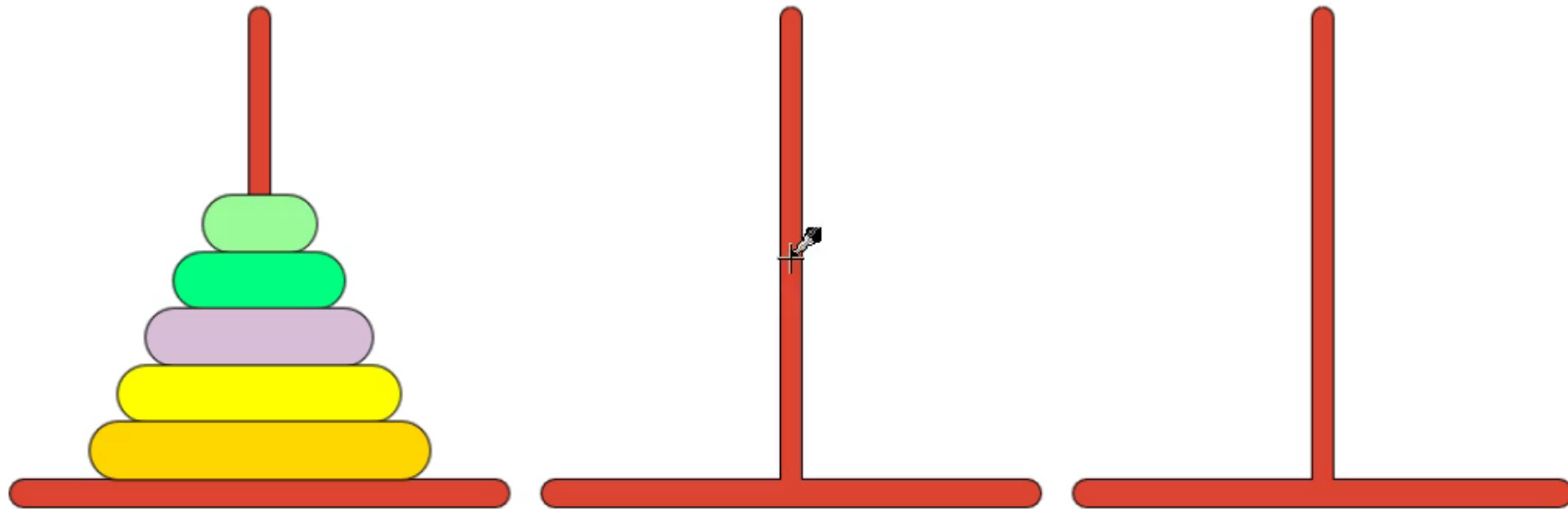
- Exponenciální složitost

- Přesně $O(2^{n+1}-1)$ tj. $O(2^n)$

- Možné heuristické řešení

- Je-li počet disků lichý/sudý, posouváme disponibilní disk na nejvzdálenější/nejbližší volnou pozici

Hanojské věže, ukázka heuristického řešení



Interaktivní hra: <https://www.mathsisfun.com/games/towerofhanoi.html>

Účinnost algoritmů

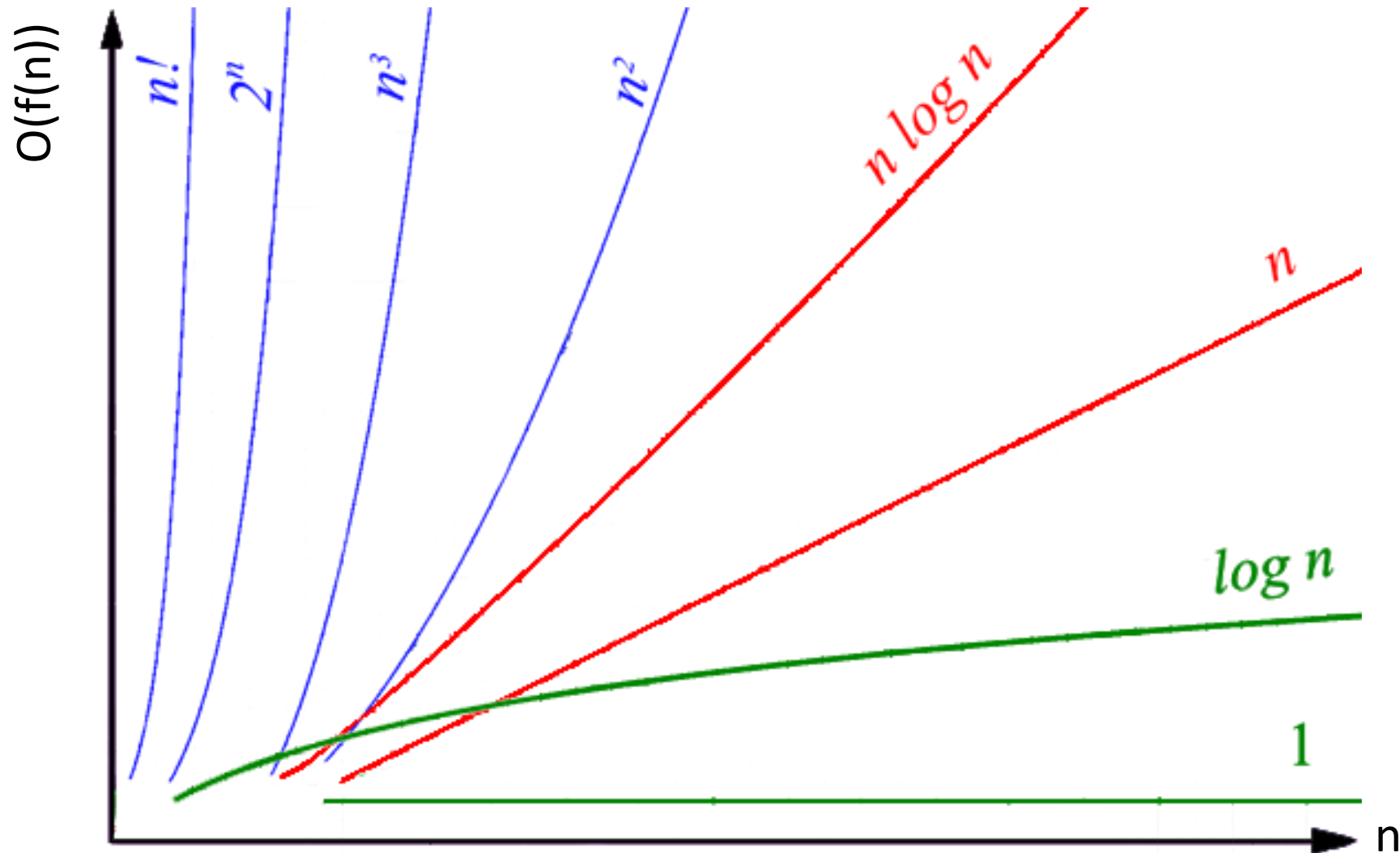
Časová a prostorová (paměťová)

- Prosté jednorázové spuštění?
- Statistika?
- Benchmarking (srovnání se standardem)?
- Sečtení instrukcí?
- Hraje roli překladač?
- A co HW?
- Jak je to se vstupními daty?

Big O notace: asymptotická složitost (nejhorší případ)

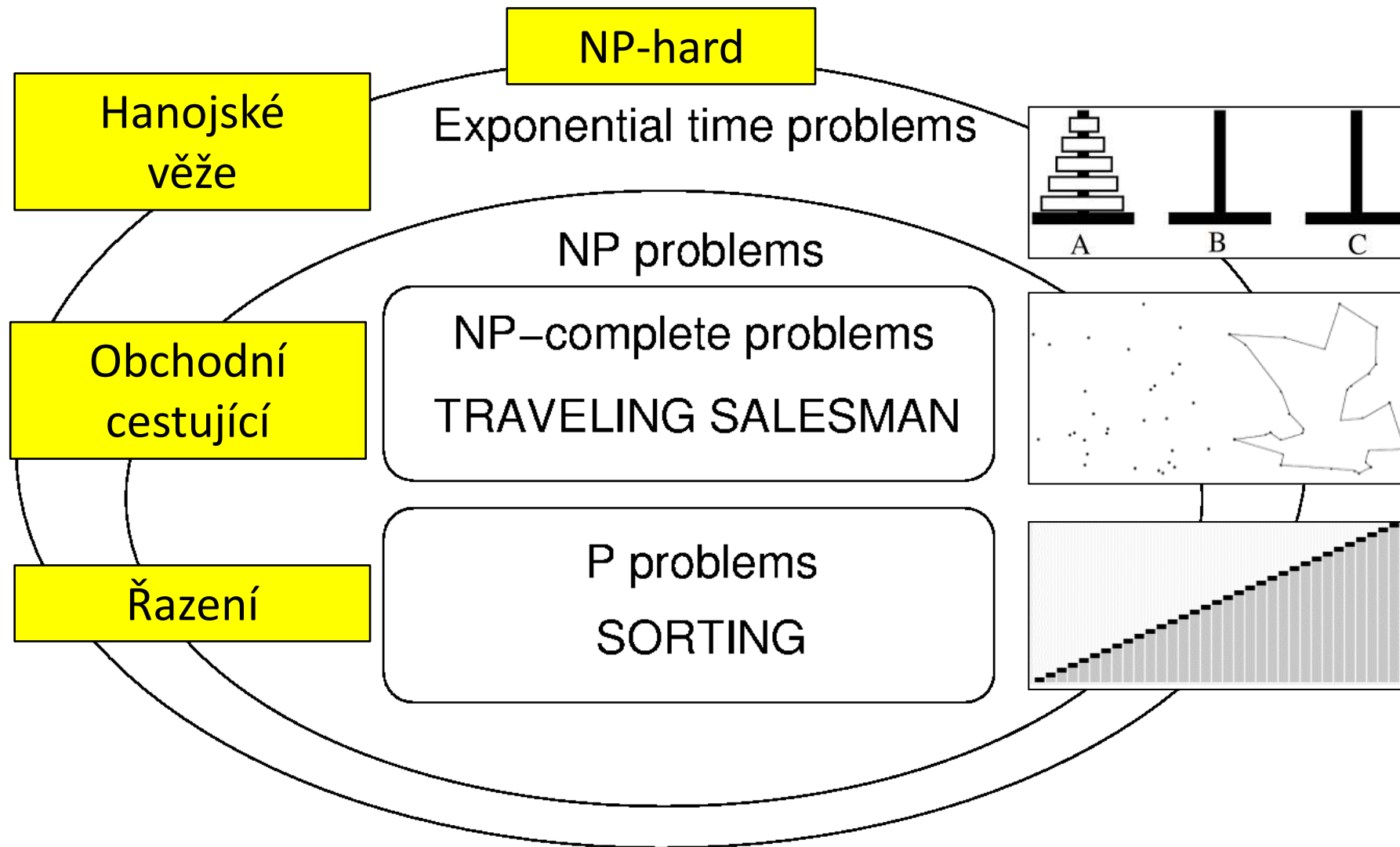
- Zanedbává aditivní a multiplikativní složky

Názorné srovnání



Zdroj: <http://http://stickybits.azurewebsites.net/wp-content/uploads/2014/11/manyCurvesTransparent.png>

Základní klasifikace algoritmů z hlediska složitosti



Modifikováno na základě: <https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/NPComplete.html>

Problémy a algoritmy

- **Výpočetní problém V**
 - úkol zpracovat / transformovat vstupní data **In** na výstupní data **Out** s požadovanými vlastnostmi
- **Algoritmus A**
 - výpočetní postup řešení problému V , čili
 - posloupnost výpočetních kroků, která vychází ze vstupní dat a vyprodukuje výstupní data požadovaných vlastností podle zadání problému
- **Instance problému**
 - problém s konkrétními daty
- **Příklad – Problém řazení čísel**
 - vstup: posloupnost čísel $In = (a_1, a_2, \dots, a_n)$
 - výstup: permutace $Out = (b_1, b_2, \dots, b_n)$ posloupnosti In taková, že platí $b_1 \leq b_2 \leq \dots \leq b_n$
- **Instance problému řazení**
 - seřadíte posloupnost čísel $In = (75, 34, 12, 88, 213, 39)$

Různé algoritmy mohou mít různou složitost

- Pro jeden problém V mohou existovat různé algoritmy $A_1, A_2, \dots$ jeho řešení
- Jednotlivé algoritmy se mohou lišit výrazně (= řádově) svými nároky na výpočetní prostředky
- Analýza složitosti algoritmu
 - výpočet nebo odhad požadavků na výpočetní prostředky, které bude provedení algoritmu vyžadovat v závislosti na velikost vstupních dat
- Velikost (složitost) vstupních dat závisí na specifikaci výpočetního problému, může to být např.
 - počet bitů vstupního čísla,
 - délka vstupní posloupnosti čísel,
 - rozměry vstupní matice,
 - počet znaků vstupního textu
 - atd.

Časová a paměťová složitost

- Časová (operační, výpočetní) složitost
 - jak závisí doba běhu algoritmu (doba výpočtu, počet provedených operací, počet provedených strojových instrukcí, ...) na velikosti vstupních dat
- Paměťová (prostorová) složitost
 - jak závisí velikost potřebné paměti (počet paměťových buněk) na velikosti vstupních dat
- Algoritmus je citlivý na vstupní data, když složitost algoritmu závisí nejen na velikosti vstupních dat, ale také na jejich hodnotách
- Proto je obecně potřeba rozlišit složitost algoritmu
 - v nejlepším případě
 - v nejhorším případě
 - v průměrném případě (nejobtížnější, protože není vždy zřejmé, co to jsou vstupní data v průměrném případě)
- Efektivita algoritmů se hodnotí především podle časové složitosti; v dalším textu proto *složitostí algoritmu* (bez přívlastku) budeme rozumět **časovou složitost algoritmu**

Příklad 1

- Najdi min a max hodnotu v poli

```
const int N = 10001, N1 = N-1;  
int a[N];
```

- verze 1

```
void minmax1() {  
    min = a[0]; max = a[0];  
    for (int i=1; i<N; i++) {  
        if (a[i]<min) min = a[i];  
        if (a[i]>max) max = a[i];  
    }  
}
```

- verze 2 – lichý počet prvků pole

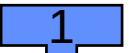
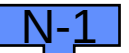
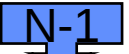
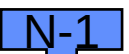
```
void minmax2() {  
    min = a[0]; max = a[0];  
    for (int i=1; i<N1; i+=2) {  
        if (a[i]<a[i+1]) {  
            if (a[i]<min) min = a[i];  
            if (a[i+1]>max) max = a[i+1];  
        } else {  
            if (a[i]>max) max = a[i];  
            if (a[i+1]<min) min = a[i+1];  
        }  
    }  
}
```

Počítání složitosti

- Elementární operace
 - aritmetická operace
 - porovnání (relační operace) nad jednoduchými typy
 - přiřazení proměnné jednoduchého typu
- Složitost můžeme počítat jako
 - celkový počet všech elementárních operací
nebo jednodušeji jako
 - celkový počet počet elementárních operací nad daty
nebo jednodušeji jako
 - celkový počet charakteristických operací, tj. porovnání čísel (znaků) v datech

Příklad 1

- Složitost minmax1 - počet všech elementárních operací

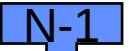
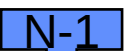
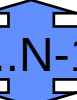
```
void minmax1() {  
      min = a[0];   max = a[0];  
      for (int i=1;   i<N;   i++) {  
          if (a[i]<min) min = a[i];  
          if (a[i]>max)   max = a[i];  
    }  
}
```

exkluzive

- Nejlepší případ:
 $1 + 1 + 1 + N + N-1 + N-1 + 0 + N-1 + 0 = 4N$
- Nejhorší případ:
 $1 + 1 + 1 + N + N-1 + N-1 + N-1 + N-1 = 5N - 1$

Příklad 1

- Složitost **minmax1** -počet elementárních operací nad daty

```
void minmax1() {  
       
    min = a[0]; max = a[0];  
        
    for (int i=1; i<N; i++) {  
           
        if (a[i]<min) min = a[i];  
             
        if (a[i]>max) max = a[i];  
    }  
}
```

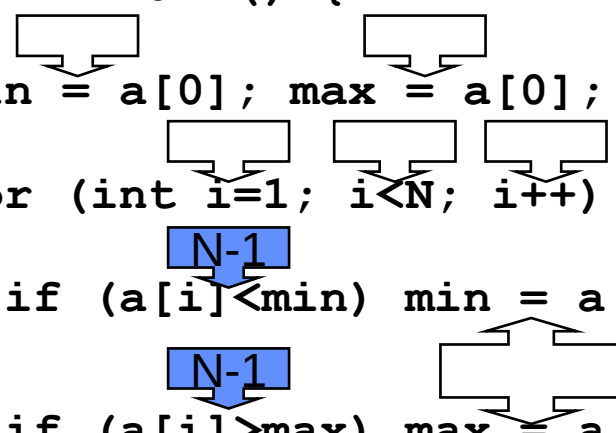
exkluzive

- Nejlepší případ:
 $1 + 1 + N-1 + 0 + N-1 + 0 = 2N$
- Nejhorší případ:
 $1 + 1 + N-1 + N-1 + N-1 = 3N - 1$

Příklad 1

- Složitost `minmax1` – pouze testy v datech

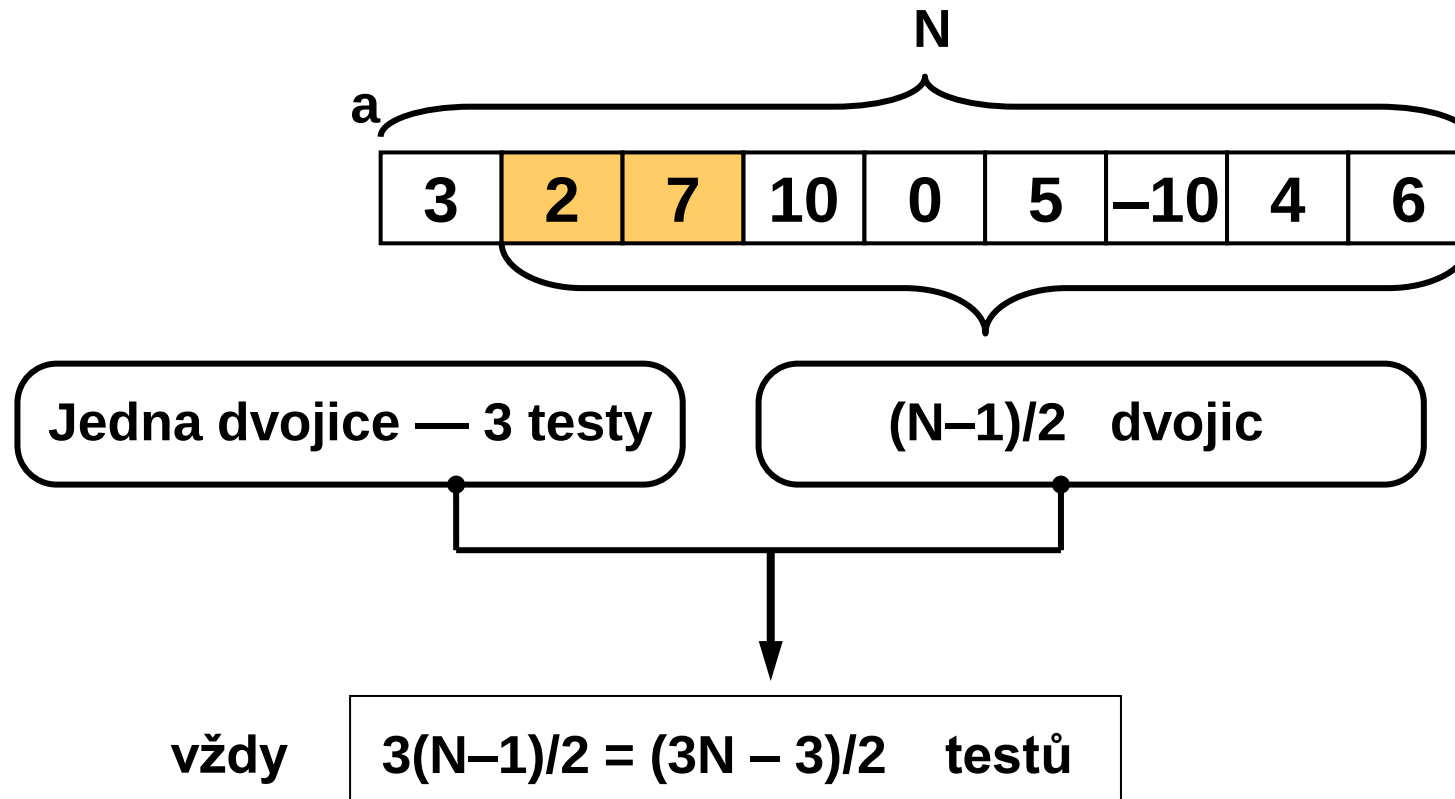
```
void minmax1() {  
    min = a[0]; max = a[0];  
    for (int i=1; i<N; i++) {  
        if (a[i]<min) min = a[i];  
        if (a[i]>max) max = a[i];  
    }  
}
```



- Nejlepší i nejhorší případ:
 $N-1 + N-1 = 2N - 2$ testů

Příklad 1

- Složitost **minmax2** – pouze testy v datech



Příklad 1 – porovnání počtu testů

počet prvků pole N	počet testů minmax1 $2N - 2$	počet testů minmax2 $(3N - 3) / 2$	poměr minmax1 / minmax2
11	20	15	1.33
21	40	30	1.33
51	100	75	1.33
101	200	150	1.33
201	400	300	1.33
501	1 000	750	1.33
1 001	2 000	1 500	1.33
2 001	4 000	3 000	1.33
5 001	10 000	7 500	1.33
1 000 001	2 000 000	1 500 000	1.33

Příklad 2

- Jsou dána dvě pole a a b délky N . Kolik prvků pole a je rovno součtu prvků pole b
- **Verze 1** – pro každý prvek pole a budeme počítat součet prvků pole b

```
int sumB() {  
    int sum = 0;  
    for (int i=0; i<N; i++)  
        sum += b[i];  
    return sum;  
}
```

```
int slow() {  
    int count = 0;  
    for (int i=0; i<N; i++)  
        if (a[i]==sumB()) count++;  
    return count;  
}
```

- Počet operací nad daty (modré podtržené): N^2

Příklad 2

- Verze 2 – nejprve vypočteme součet prvků pole *b* a pak porovnáváme prvky pole *a* s vypočteným součtem

```
int sumB() {  
    int sum = 0;  
    for (int i=0; i<N; i++)  
        sum += b[i];  
    return sum;  
}  
  
int fast() {  
    int count = 0, sum = sumB();  
    for (int i=0; i<N; i++)  
        if (a[i]==sum) count++;  
    return count;  
}
```

- Počet operací nad daty (modré podtržené): $N + N = 2N$

Příklad 2 – porovnání počtu operací

počet prvků polí N	slow počet operací N^2	fast počet operací $2N$	poměr slow / fast
11	121	22	5.5
21	441	42	10.5
51	2 601	102	25.5
101	10 201	202	50.5
201	40 401	402	100.5
501	251 001	1 002	250.5
1 001	1 002 001	2 002	500.5
2 001	4 004 001	4 002	1 000.5
5 001	25 010 001	10 002	2 500.5
1 000 001	1 000 002 000 001	2 000 002	500 000.5

Příklad 3

- Hledání prvku pole s danou hodnotou

- Verze 1 – sekvenční hledání

```
int sekvenčni(int x) {  
    for (int i=0; i<N; i++)  
        if (a[i]==x) return i;  
    return -1;  
}
```

- Počet testů nad daty:
 - v nejlepším případě 1
 - v nejhorším případě N
 - v průměrném případě $(N+1)/2$

Příklad 3

- Verze 2 – v seřazeném poli binární hledání (půlením intervalu)

```
int binarni(int x) {  
    int d=0, h=N-1, k;  
    while (d<=h) {  
        k = (h-d)/2;  
        if (a[k]<x) d = k+1;  
        else if (a[k]>x) h = k-1;  
        else return k;  
    }  
    return -1;  
}
```

- Počet testů nad daty:
 - v nejlepším případě 2
 - v nejhorším případě $2 \log_2 N$
 - v průměrném případě $1 + \log_2 N$???
- Poznámka: dělíme-li N opakovaně 2, pak po $\lceil \log_2 N \rceil$ krocích dostaneme číslo menší nebo rovné 1

Příklad 3 – porovnání počtu operací

počet prvků pole N	sekvenční počet operací $(N+1)/2$	binární počet operací $1 + \log_2 N$	poměr sekvenční / binární
5	3	4	0.75
10	6	5	1.20
20	11	6	1.83
50	25	7	3.71
100	51	8	6.38
200	101	9	11.22
500	251	10	25.10
1 000	501	11	45.55
2 000	1 001	12	83.42
5 000	2 501	14	178.64
1 000 000	500 001	21	23 809.57

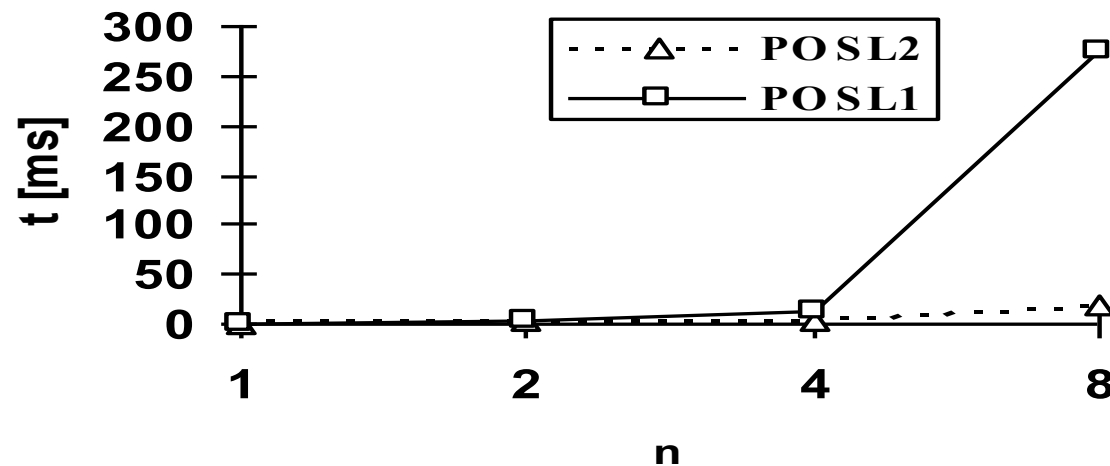
Příklad4

- **Fibonnacciho posloupnost**

$$f(0) = 0, \quad f(1) = 1, \quad f(n) = f(n-1) + f(n-2)$$

Př: 0, 1, 1, 2, 3, 5, 8, 13, 21....

- Nerekurzivní výpočet (POSL2)
- Rekurzivní výpočet (POSL1)



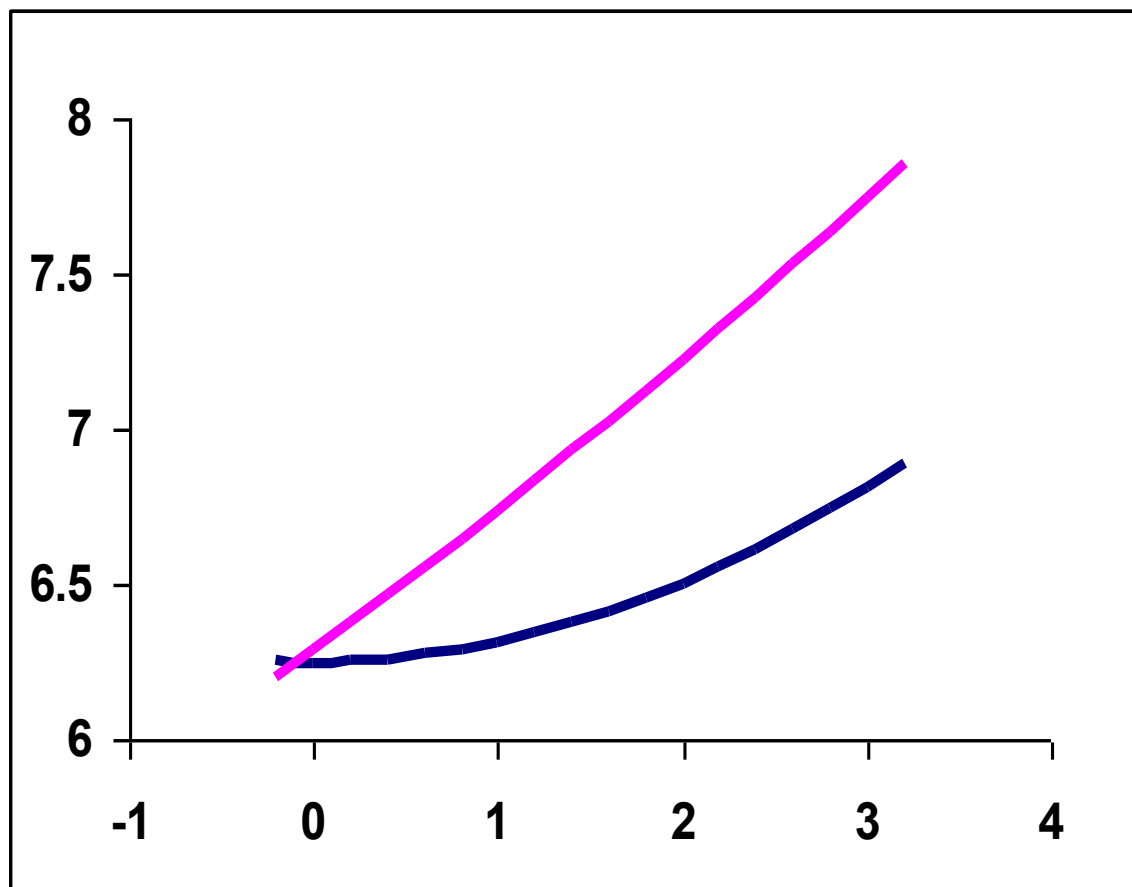
Časová složitost algoritmů

- Tabulka udávající dobu výpočtu pro různé časové složitosti za předpokladu, že 1 operace trvá 1 μs

	n					
	10	20	40	60	500	1000
$\log_2 n$	3,3 μs	4,3 μs	5 μs	5,8 μs	9 μs	10 μs
n	10 μs	20 μs	40 μs	60 μs	0,5ms	1ms
$n \log_2 n$	33 μs	86 μs	0,2ms	0,35ms	4,5ms	10ms
n^2	0,1ms	0,4ms	1,6ms	3,6ms	0,25s	1s
n^3	1ms	8ms	64ms	0,2s	125s	17min
n^4	10ms	160ms	2,56s	13s	17h	11,6dní
2^n	1ms	1s	12,7 dní	36000 let	10^{137} let	10^{287} let
$n!$	3,6s	77000 let	10^{34} let	10^{68} let	10^{1110} let	10^{2554} let

Růst funkcí

- Porovnejme růst dvou funkcí

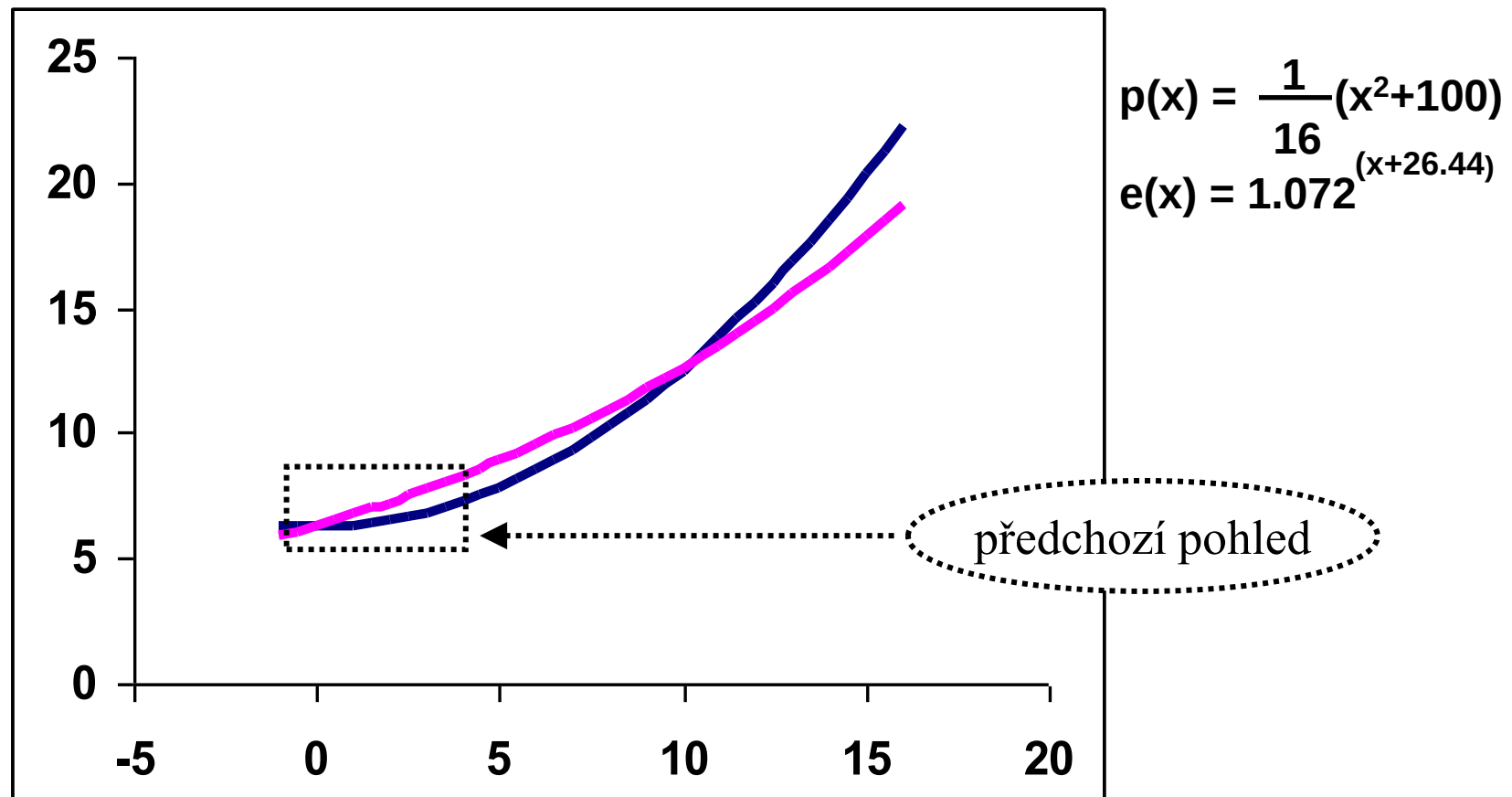


$$e(x) = 1.072^{(x+26.44)}$$

$$p(x) = \frac{1}{16}(x^2+100)$$

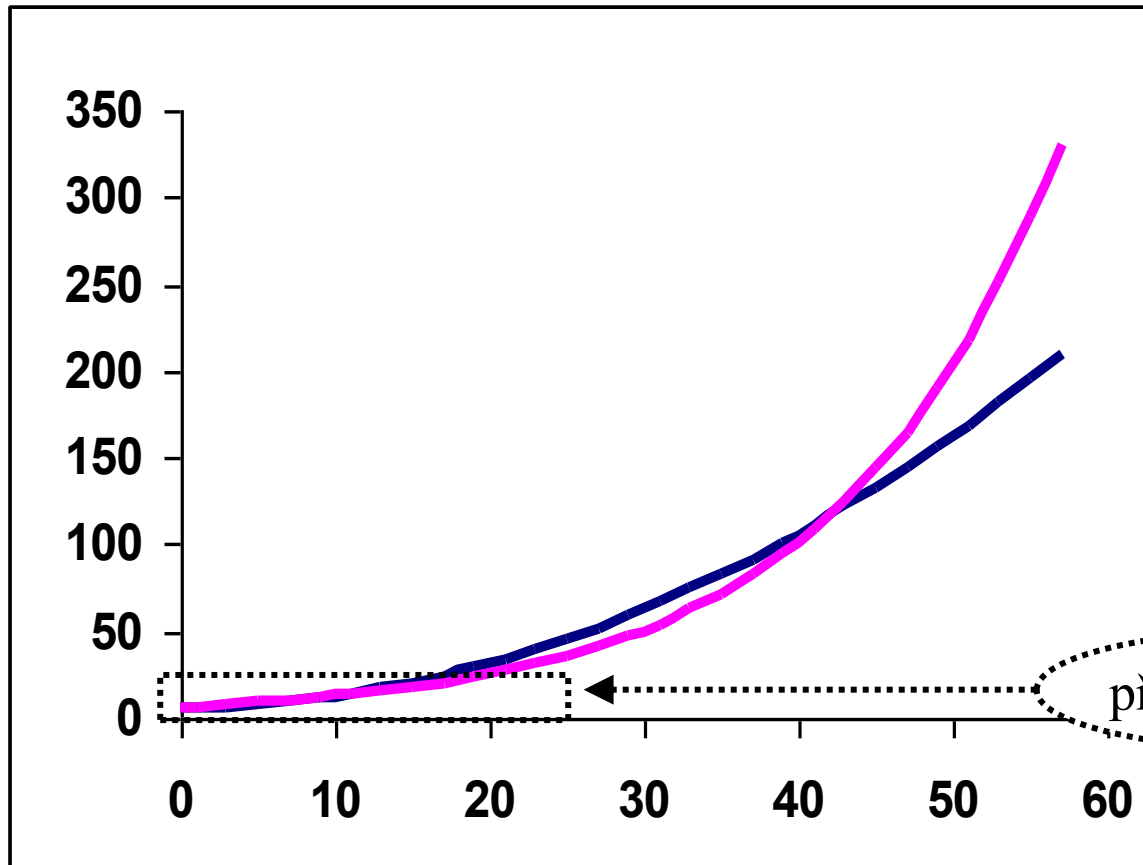
Růst funkcí

- Porovnejme růst dvou funkcí



Růst funkcí

- Porovnejme růst dvou funkcí



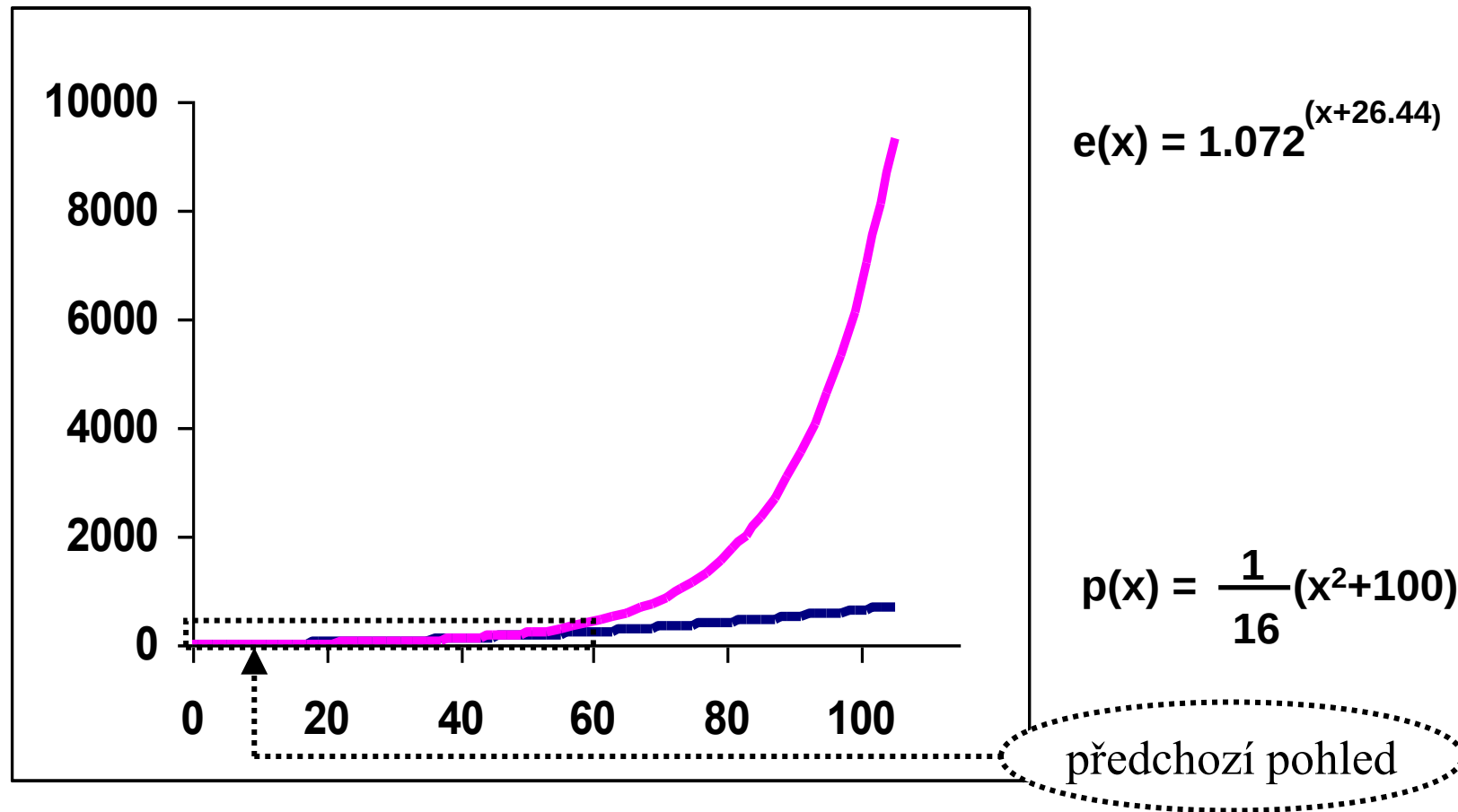
$$e(x) = 1.072^{(x+26.44)}$$

$$p(x) = \frac{1}{16}(x^2+100)$$

předchozí pohled

Růst funkcí

- Porovnejme růst dvou funkcí



Asymptotická složitost

- Přesné určení složitosti algoritmů
 - lze zvládnout jen pro jednoduché algoritmy
 - pro složité algoritmy je to velmi obtížné
 - proto se to typicky nedělá
- Asymptotická složitost algoritmů
 - vyjadřuje, jak složitost algoritmu závisí na velikosti vstupních dat v limitě, tj. když velikost instance problému roste nade všechny meze
 - vyjadřuje řád růstu funkce složitosti se zanedbáním příspěvku nižšího řádu
 - zanedbává multiplikativní konstantu

Asymptotická horní mez

- **O-notation** (Omikron – Big O)

- Relační definice:

Funkce $f(n)$ je **nejvýše řádu** $g(n)$, psáno $f(n) = O(g(n))$, jestliže

$$\exists c > 0, n_0 > 0, \forall n \geq n_0 : f(n) \leq c g(n)$$

- Množinová definice:

Pro funkci $g(n)$ je $\Theta(g(n))$ množina funkcí $f(n)$, pro které platí

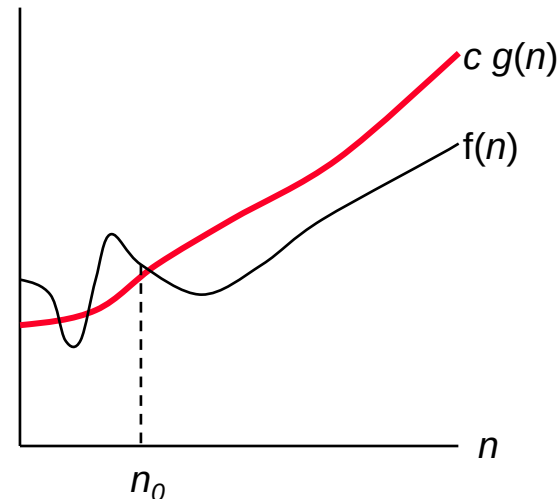
$$\{f(n) : \exists c > 0, n_0 > 0, \forall n \geq n_0 : f(n) \leq c g(n)\}$$

Zápisy

$$f(n) = O(g(n))$$

$$f(n) \in O(g(n))$$

jsou rovnocenné



Asymptotická horní mez

- Slovní interpretace zápisu $f(n) = O(g(n))$
 - $f(n)$ neroste rychleji (není horší), než $g(n)$
 - horší to být nemůže
- Zápisem $f(n) = O(g(n))$ vyjadřujeme, že $g(n)$ je pro $f(n)$ asymptotickou horní mezí **stejného nebo vyššího řádu**
- Příklad:
$$3n^2 - 5n - 15 = O(n^2)$$
ale také
$$3n^2 - 5n - 15 = O(n^3)$$
- O-výrazy používáme pro odhady složitosti algoritmů v **nejhorších případech**

Asymptotická dolní mez

- **Ω -notace** (Omega)

- Relační definice:

Funkce $f(n)$ je **nejméně řádu** $g(n)$, psáno $f(n) = \Omega(g(n))$, jestliže

$$\exists c > 0, n_0 > 0, \forall n \geq n_0 : f(n) \geq c g(n)$$

- Množinová definice:

Pro funkci $g(n)$ je $\Omega(g(n))$ množina funkcí $f(n)$, pro které platí

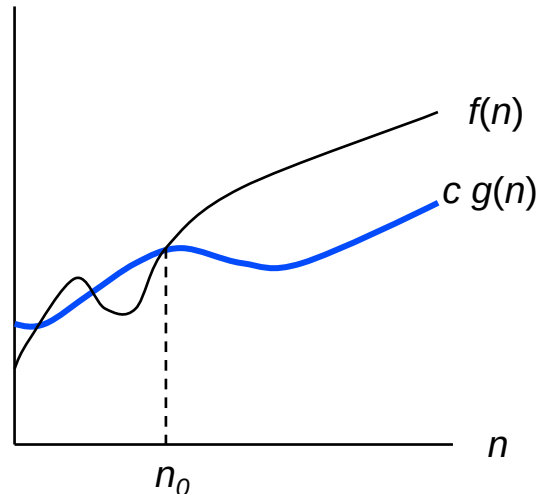
$$\{f(n) : \exists c > 0, n_0 > 0, \forall n \geq n_0 : f(n) \geq c g(n)\}$$

Zápisy

$$f(n) = \Omega(g(n))$$

$$f(n) \in \Omega(g(n))$$

jsou rovnocenné



Asymptotická dolní mez

- Slovní interpretace zápisu $f(n) = \Omega(g(n))$
 - $f(n)$ neroste pomaleji (není lepší), než $g(n)$
 - lepší to být nemůže
- Zápisem $f(n) = \Omega(g(n))$ vyjadřujeme, že $g(n)$ je pro $f(n)$ asymptotickou dolní mezí **stejného nebo nižšího** řádu
- Příklad:
 $3n \log n + 3\sqrt{n} + 1 = \Omega(n \log n)$
ale také
 $3n \log n + 3\sqrt{n} + 1 = \Omega(n)$
- Ω -výrazy používáme pro odhady složitosti algoritmů v **nejlepších případech**

Asymptoticky těsná mez

- **Θ-notace** (Theta)

- Relační definice:

Funkce $f(n)$ je **téhož řádu jako** funkce $g(n)$, psáno $f(n) = \Theta(g(n))$, jestliže

$$\exists c_1 > 0, c_2 > 0, n_0 > 0, \forall n \geq n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n)$$

- Množinová definice:

Pro funkci $g(n)$ je $\Theta(g(n))$ množina funkcí $f(n)$, pro které platí

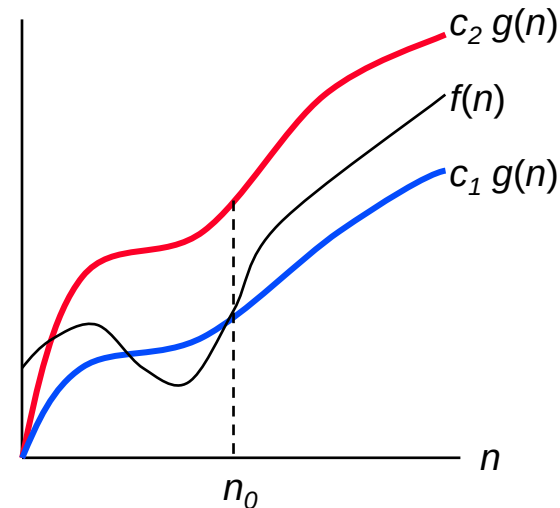
$$\{f(n) : \exists c_1 > 0, c_2 > 0, n_0 > 0, \forall n \geq n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n)\}$$

Zápisy

$$f(n) = \Theta(g(n))$$

$$f(n) \in \Theta(g(n))$$

jsou rovnocenné



Asymptoticky těsná mez

- Slovní interpretace zápisu $f(n) = \Theta(g(n))$
 - $f(n)$ roste stejně rychle (není horší ani lepší), jako $g(n)$
 - lepší ani horší to být nemůže
- $f(n) = \Theta(g(n))$ implikuje $f(n) = O(g(n))$
(Θ je silnější podmínka než O , neboli $\Theta(g(n))$ je podmnožinou $O(g(n))$)
- $f(n) = \Theta(g(n))$ implikuje $f(n) = \Omega(g(n))$
(Θ je silnější podmínka než Ω , neboli $\Theta(g(n))$ je podmnožinou $\Omega(g(n))$)
- Aby platilo, že, že $f(n) = \Theta(g(n))$, musí platit
 $f(n) = O(g(n))$ a současně $f(n) = \Omega(g(n))$

Pravidla pro určování řádu růstu funkce

- $\Theta(f) + \Theta(g) = \Theta(\max(f, g))$
- $\Theta(f) + \Theta(g) = \Theta(f + g)$
- $\Theta(f) \cdot \Theta(g) = \Theta(f \cdot g)$
- jestliže $g(n) \leq f(n)$ pro všechna $n \geq n_0$, $n_0 > 1$, pak
 $\Theta(f) + \Theta(g) = \Theta(f)$
- $\Theta(c \cdot f(n)) = \Theta(f(n))$, c je konstanta
- $f(n) = \Theta(f(n))$

Statisticky průměrná složitost

Složitost v průměrném případě $T_{\text{stř}}(n)$

- $V = \{v_i\}, i = 1, 2, \dots, n$ - prostor možných řešení algoritmu A,
 $p(v_i)$ - pravděpodobnost, že nastane řešení v_i ,
 T_i - časová složitost i-tého řešení.
průměrná časová složitost algoritmu A je

$$T_{\text{stř}}(n) = \sum T_i \cdot p(v_i)$$

- Příklad:
Vypočítejte statisticky průměrnou složitost algoritmu hledání prvku x v poli
 1. sekvenční metodou,
 2. metodou binárního půlení.

Součty řad

- **Aritmetická řada:** $a_{k+1} = a_k + d$, $k = 1, 2, \dots$, difference $d \in \mathbb{R}$

Částečný součet

$$S_n = \sum_{k=1}^n a_k = \frac{n}{2} \cdot (a_1 + a_n) = n \cdot a_1 + \frac{n \cdot (n-1)}{2} \cdot d$$

- **Geometrická řada:** $a_{k+1} = a_k \cdot q$, q reálné, $k = 1, 2, \dots$

Částečný součet pro $q \neq 1$

$$S_n = \sum_{k=1}^n a_k = a_1 \cdot (1 + q + q^2 + \dots + q^{n-1}) = a_1 \sum_{k=0}^{n-1} q^k = a_1 \frac{q^n - 1}{q - 1}$$

Pro $|q| < 1$ je geometrická řada konvergentní

$$S_\infty = a_1 / (1 - q)$$